DAY-6 SQL

1. What is SQL?

Definition

SQL (Structured Query Language) is a standard language used to create, store, retrieve, update, and manage data in a relational database.

Uses of SQL

- Create databases and tables
- Insert records
- Retrieve records
- Update records
- Delete records
- Manage databases

Example

```
SELECT * FROM Student;
```

---

2. Where is SQL Used?

SQL is used wherever relational databases are used.

Applications

- Banking Systems
- Hospital Management Systems
- School & College Management
- Railway Reservation Systems
- Airline Reservation Systems
- E-Commerce Websites
- Social Media Applications

- Employee Management Systems
- Library Management System

---

3. What is an Operator?

Definition

An **operator** is a symbol or keyword used to perform an operation on one or more operands.

Example

```
SELECT * FROM Student
WHERE Marks > 80;
```

Here

> is an operator.

---

4. Types of SQL Operators

There are **5 major types of operators**.

- **Arithmetic Operators**

Definition

Used to perform mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example

```
SELECT Salary + 5000
FROM Employee;
```

- **Comparison Operators**

Definition

Used to compare two values.

Operator	Meaning
=	Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal
<>	Not Equal
!=	Not Equal

Example

```
SELECT *
FROM Student
WHERE Marks > 80;
```

- **Logical Operators**

Definition

Logical operators combine multiple conditions.

AND

Returns records only if **all conditions are true**.

Example

```
SELECT *
FROM Student
WHERE Branch='CSE'
AND Marks>80
AND Age=20;
```

OR

Returns records if **any one condition is true**.

Example

```
SELECT *
FROM Student
WHERE Branch='CSE'
OR Marks>80;
```

NOT

Reverses the condition.

Example

```
SELECT *
FROM Student
WHERE NOT Branch='CSE';
```

- **Special Operators**

IN

Definition

Used to check whether a value exists in a list.

Example

```
SELECT *
FROM Student
WHERE Branch IN ('CSE','ECE');
```

BETWEEN

Definition

Used to select values within a specified range.

Example

```
SELECT *
FROM Student
WHERE Marks BETWEEN 70 AND 90;
```

LIKE

Definition

Used for **pattern matching**.

Example

```
SELECT *
FROM Student
WHERE Name LIKE 'A%';
```

IS NULL

Definition

Used to find NULL values.

Example

```
SELECT *
FROM Student
WHERE Phone IS NULL;
```

EXISTS

Definition

Returns TRUE if a subquery returns one or more rows.

Example

```
SELECT Name
FROM Student
```

```
WHERE EXISTS
(
SELECT *
FROM Student
WHERE Marks>90
);
```

- **Set Operators**

UNION

Definition

Combines two or more SELECT statements and removes duplicate rows.

Example

```
SELECT Name FROM Student1
UNION
SELECT Name FROM Student2;
```

UNION ALL

Definition

Combines two or more SELECT statements and keeps duplicate rows.

Example

```
SELECT Name FROM Student1
UNION ALL
SELECT Name FROM Student2;
```

INTERSECT

Definition

Returns only the common rows present in both SELECT statements.

Example

```
SELECT Name FROM Student1
INTERSECT
SELECT Name FROM Student2;
```

Difference

UNION	UNION ALL
Removes duplicates	Keeps duplicates
Returns unique rows	Returns all rows
Slower	Faster

- **Pattern Matching in SQL**

Definition

Pattern Matching is a technique used in SQL to search for data that matches a specified pattern. It is performed using the **LIKE** operator along with **wildcard characters**.

Wildcard Characters Used in Pattern Matching

There are **2 wildcard characters** used with the LIKE operator:

Wildcard	Meaning
%	Matches **zero, one, or more characters**
_	Matches **exactly one character**

- **Types of Pattern Matching**

There are 7 commonly used pattern matching types in SQL.

Type	Pattern	Meaning
1	A%	Starts with A
2	%A	Ends with A
3	%A%	Contains A anywhere
4	_A%	Second character is A
5	A_	Starts with A and has exactly 2 characters
6	_____	Exactly 5 characters
7	A%n	Starts with A and ends with n

---

1. **Starts With (A%)**

Meaning: Finds values that start with A.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE 'A%';
```

Example Results:

- Amit
- Arun
- Ananya

---

2. **Ends With (%A)**

Meaning: Finds values that end with A.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE '%A';
```

Example Results:

- Priya
- Sneha
- Asha

3**. Contains (%A%)**

Meaning: Finds values containing A anywhere in the text.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE '%A%';
```

Example Results:

- Rahul
- Karan
- Ananya

---

4. **Second Character (_A%)**

Meaning: Finds values where the second character is A.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE '_A%';
```

Example Results:

- Rahul
- Ramesh
- Karan

5**. Exactly Two Characters (A_)**

Meaning: Finds values that start with A and have exactly 2 characters.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE 'A_';
```

Example Results:

- An
- Ab
- Ar

6**. Exactly Five Characters (_____)**

Meaning: Finds values with exactly 5 characters.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE '_____';
```

Example Results:

- Rahul
- Kiran

7**. Starts With A and Ends With n (A%n)**

Meaning: Finds values that start with A and end with n.

Example Query:

```
SELECT * FROM Student
WHERE Name LIKE 'A%n';
```

Example Results:

- Arun
- Aman

- **Aggregate Functions**

Definition

Aggregate functions perform calculations on multiple rows and return a single value.

Function	Purpose
COUNT()	Counts rows
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

Examples

SELECT COUNT(*) FROM Student;

SELECT SUM(Salary) FROM Employee;

SELECT AVG(Marks) FROM Student;

SELECT MIN(Marks) FROM Student;

SELECT MAX(Marks) FROM Student;

---

1. COUNT()

Definition

COUNT() returns the total number of rows.

Example Table

Student
Rahul

Student
Priya
Sneha
Arjun

Query

```
SELECT COUNT(*)
FROM Student;
```

Output

4

Explanation:
There are **4 students**, so COUNT() returns **4**.

---

2. SUM()

Definition

SUM() returns the total of all numeric values in a column**.**

Table

Marks
80
90
70
60

Query

```
SELECT SUM(Marks)
FROM Student;
```

Calculation

80 + 90 + 70 + 60 = 300

Output

300

---

3. AVG()

Definition

AVG() returns the average (mean) of all numeric values.

Query

```
SELECT AVG(Marks)
FROM Student;
```

Calculation

(80 + 90 + 70 + 60) ÷ 4 = 75

Output

75

---

4. MIN()

Definition

MIN() returns the smallest value in a column.

Query

```
SELECT MIN(Marks)
FROM Student;
```

Output

60

Explanation:
The smallest mark is **60**.

---

5. MAX()

Definition

MAX() returns the largest value in a column.

Query

```
SELECT MAX(Marks)
FROM Student;
```

Output

90

Explanation:
The highest mark is **90**.

4.WHERE

Definition

Filters **individual rows** before grouping.

Example

```
SELECT *
FROM Employee
WHERE Salary>50000;
```

---

5. HAVING

Definition

Filters **groups** after **GROUP BY** using aggregate functions.

Example

```
SELECT Department,AVG(Salary)
FROM Employee
```

```
GROUP BY Department
HAVING AVG(Salary)>50000;
```

- **Difference Between WHERE and HAVING**

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions directly	Can use aggregate functions

---

6. INNER JOIN

Definition

INNER JOIN combines rows from two tables based on a common column and returns only the matching records.

Set Formula

$A \cap B$

(Common records)

Syntax

```
SELECT *
FROM Student
INNER JOIN Marks
ON Student.StudentID=Marks.StudentID;
```

Example

Student Table

StudentID	Name
101	Rahul

StudentID	Name
102	Priya
103	Sneha

Marks Table

StudentID	Marks
101	85
103	90
104	78

Output

StudentID	Name	Marks
101	Rahul	85
103	Sneha	90

Only the matching records are returned.

MICROSERVICES

- **What are Microservices?**

Microservices is a way of developing a software application by dividing it into **small, independent services**. Each service performs **one specific task** and communicates with other services through **APIs**.

Simple Real-Life Example

Imagine a **hospital**.

There isn't just one doctor doing everything.

- 👨‍💼 Reception → Registers patients

- 👩‍⚕️ Doctor → Examines patients
- 💊 Pharmacy → Gives medicines
- 💳 Billing → Collects payment

Each department has **one responsibility**.

Similarly, in **Microservices**, each service has **one job**.

Example Project: Online Shopping Website

Suppose you build an Amazon-like website.

Instead of writing one big application, you divide it into services.

User Service
↓
Product Service
↓
Cart Service
↓
Order Service
↓
Payment Service
↓
Notification Service

What Does Each Service Do?

1. User Service

Handles:

- Login
- Registration
- User Profile

---

2. Product Service

Handles:

- Product details
- Search products
- Categories

---

3. Cart Service

Handles:

- Add to cart
- Remove from cart
- Update quantity

---

4. Order Service

Handles:

- Place order
- Cancel order
- Track order

---

5. Payment Service

Handles:

- UPI payment
- Credit card
- Debit card
- Wallet payment

---

6. Notification Service

Handles:

- Email
- SMS
- Order confirmation

How Do They Work Together?

Suppose a customer buys a mobile phone.

Step 1

User logs in.

➞ **User Service** checks the login.

↓

Step 2

Customer searches for a mobile.

➞ **Product Service** shows products.

↓

Step 3

Customer adds the mobile to the cart.

➞ **Cart Service** stores the cart items.

↓

Step 4

Customer places the order.

➞ **Order Service** creates the order.

↓

Step 5

Customer pays.

➞ **Payment Service** processes the payment.

↓

Step 6

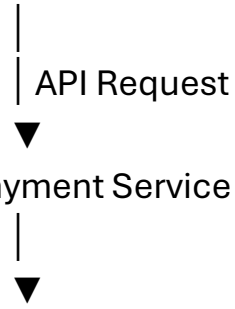
Payment is successful.

➞ **Notification Service** sends a confirmation email or SMS.

How Do Services Communicate?

They communicate using **APIs**.

Example:

Order Service


Payment Service

Payment Successful

The **Order Service** asks the **Payment Service** to process the payment through an API.

Why Do We Use Microservices?

Imagine your application has **100 features**.

If everything is in one program:

- Difficult to understand
- Difficult to update
- Difficult to fix errors

If it is divided into microservices:

- Each service is small.
- Easy to develop.
- Easy to test.
- Easy to update.
- Easy to scale.

Advantages

- Easy to develop
- Easy to maintain
- Easy to update
- Each service works independently
- Different teams can work on different services

Disadvantages

- More complex than a small application
- More APIs to manage
- Harder to debug because many services communicate

Microservices vs Monolithic

Monolithic	Microservices
One large application	Many small services
One deployment	Each service deployed separately

Monolithic	Microservices
Hard to maintain	Easy to maintain
Hard to scale	Easy to scale
One codebase	Multiple codebases

Simple example: Think of **Amazon**:

Login → User Service
Products → Product Service
Cart → Cart Service
Orders → Order Service
Payment → Payment Service
Email/SMS → Notification Service

One Service = One Responsibility

This is the main idea of **Microservices**.